# 并查集 最小生成树

# I. 并查集(Union-Find Tree)

在开始正式的介绍并查集之前，我们还需要说几个图论的名词：

1. 联通图： 图中任意两个顶点间都存在路径可以互相到达的图.
2. 联通分量：图中所有可以互相到达的点划分在同一组时，所形成的独立子图，我们称这个子图叫一个联通分量.
3. 邻接： 如果顶点 $u$ 和顶点 $v$ 之间存在直接相连的边，我们称这两个顶点是 邻接 的.

让我们以一个例题，来引入我们今天要讲的第一个东西吧！

对于一个具有 $N$ 个顶点的图，请快速处理以下两种查询:

- 查询 1：添加连接顶点 $u$ 和 $v$ 的双向边.
- 查询 2：判断顶点 $u$ 和 $v$ 是否属于同一联通分量.

其中，假设图初始状态无边且需要处理 $Q$ 次查询.

输入格式：第一行两个数字 $N, Q$，接下来 $Q$ 行，每行有两种情况:

对于查询一，以 1 $u$ $v$ 的形式，查询二，以 2 $u$ $v$ 的形式.

输出格式：对于每一次查询二，输出一行一个数字，表示答案.

## I.I. 什么是 Union-Find (也称为 DSU(disjoint set union))

这是一种可有效管理分组的数据结构。具体来说，可快速处理以下两种查询：

合并查询：将包含元素 $u$ 的组和包含元素 $v$ 的组进行合并.

查找查询：判断元素 $u$ 和元素 $v$ 是否属于同一组。

我们发现，合并查询对应与题中的查询 1, 而查找查询则对应查询 2。

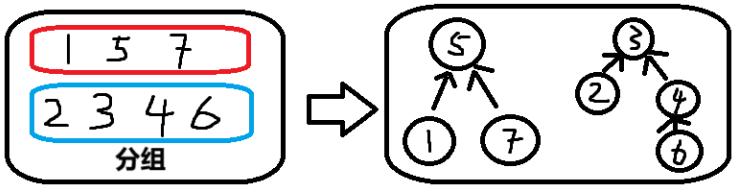
## I.II. Union-Find 的结构

那么 Union-Find 是如何处理数据的呢？简而言之，他采用满足以下条件的有根树的结构:

同一组内的顶点有相同的根节点

不同组的顶点有不同的根节点

例如，当整数 $1-7$ 被划分为 $\{1,5,7\}$ 和 $\{2,3,4,6\}$ 两组时，其对应的 Union-Find 结构示例如下图所示.

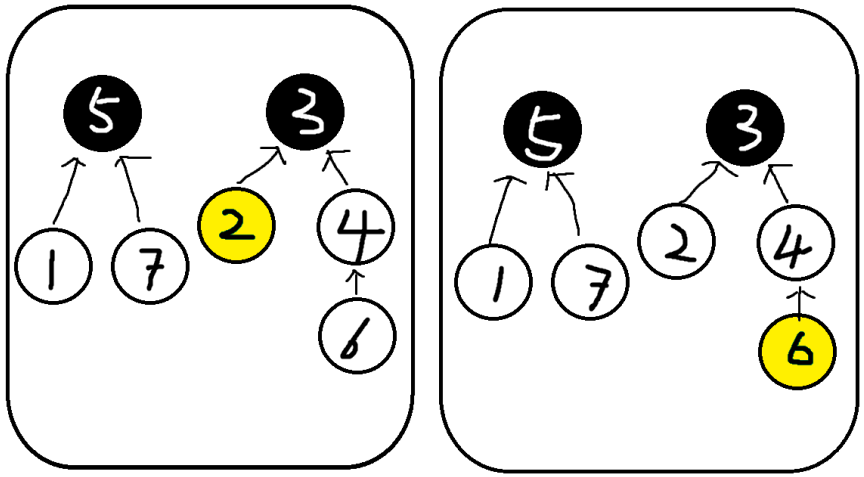


## I.III. 处理查找查询

首先介绍查找查询的处理方法。当用 $\texttt{root}(x)$ 表示顶点 $x$ 的根节点时，可通过以下方式判断元素 $u$ 与 $v$ 是否属于同一组：

- 如果 $\texttt{root}(u) = \texttt{root}(v)$ : 同组.
- 如果 $\texttt{root}(u) \neq \texttt{root}(v)$ : 不同组.

在下图的案例中，元素 2 和 6 属于同一组，因为顶点 2 和 6 的根节点均为顶点 3，即满足 $\texttt{root}(2) = \texttt{root}(6)$.




左图顶点 2 的根节点为顶点 3，右图顶点 6 的根节点为顶点 3.

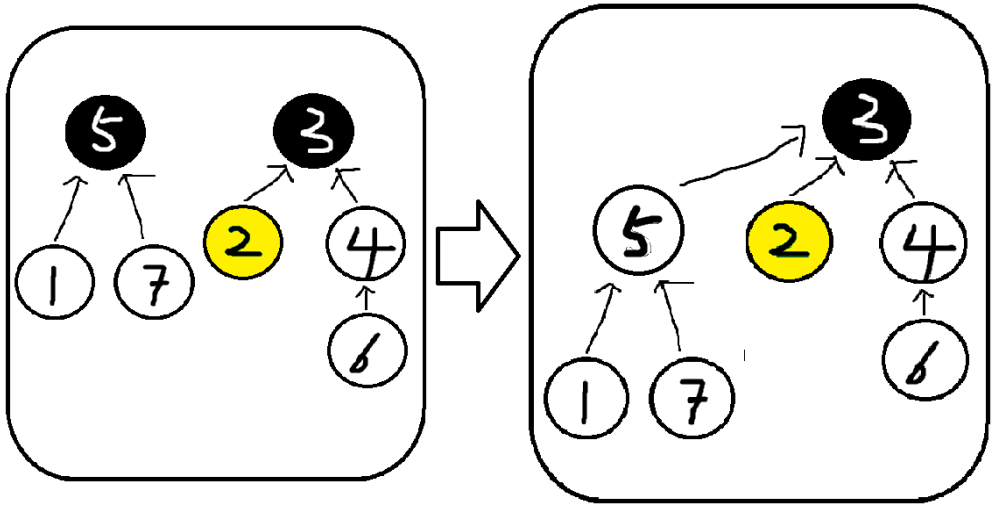
## I.IV. 处理合并查询

接下来处理合并查询，其核心操作是「连接两个数的根节点」.以下图为例，只需要将顶点 5 的父节点设为顶点 3 即可完成合并.

说的严谨一些，当用 $par[x]$ 表示顶点 $x$ 的父节点时，可通过以下步骤合并「包含顶点 $u$ 的组」与「包含顶点 $v$ 的组」：

- 令 $par[\texttt{root}(u)] = \texttt{root}(v)$.

也就是说，包含顶点 $v$ 的组的根节点将成为新组的根。当然，反向操作 $par[\texttt{root}(v)] = \texttt{root}(u)$ 同样有效.



## I.V. 简单 Union-Find 的时间复杂度

简单评估一下 Union-Find 的时间复杂度. 如前所述，Union-Find 的核心是求顶点 $x$ 根节点的函数 root($x$)，其代码如下，计算次数与「顶点 $x$ 到根节点的距离」成正比！

```cpp
//返回顶点x根节点的函数，对于没有父节点的顶点，记录“par[x] = -1”
int root(int x) {
  while(true) {
    if(par[x] == -1) break; //如果没有父亲，那么这就是根节点
    x = par[x]; //前进到其父节点
  }
  return x;
}
```

我们考虑最差的情况，合并操作可能导致最长的这样的路径达到 $N-1$，此时 root($x$) 函数需要 $O(N)$ 的时间复杂度！
那么我们可不可以优化它呢？

## I.VI. 改良 Union-Find!

那么，我们考虑如何才能优化他的时间复杂度呢？

事实上，只需采用将规模更大的组置于上层的策略（被称为 Union by Size），它可以将复杂度从 $O(N)$ 降至 $O(\log N)$.

还有一种优化被称为「路径压缩」，即查询过程中经过的每个元素都属于该集合，我们可以将其直接连到根节点以加快后续查询. 同样是 $O(\log N)$.

**P.S.:**事实上，结合以上的两种写法可以将时间复杂度降低到 $O(\alpha(N))$. 其中 $\alpha(N)$ 是阿克曼函数的反函数，即使 $N = 10^5$, 他也不会超过 4.

最后 Union by Size 算法可以通过以下变量实现：

- $siz[x]$ :「以顶点 $x$ 为根的数」的顶点数.

在合并处理查询时，需要更新 $siz[x]$ 的值.

接下来给出使用 Union by Size 的代码.

```cpp
#include <bits/stdc++.h>
using namespace std;
struct UnionFind {
  int par[100009], siz[100009];
  void init(int N) {
    for(int i = 1; i <= N; i ++) par[i] = -1;
    for(int i = 1; i <= N; i ++) siz[i] = 1;
  }
  int root(int x) {
    while(true) {
      if(par[x] == -1) break;
      x = par[x];
    }
    return x;
  }
  void unite(int u, int v) {
    int RootU = root(u);
    int RootV = root(v);
    if(RootU == RootV) return;
    if(siz[RootU] < siz[RootV]) {
      par[RootU] = RootV;
      siz[RootV] = siz[RootU] + siz[RootV];
    } else {
      par[RootV] = RootU;
```

```
      siz[RootU] = siz[RootU] + siz[RootV];
    }
  }
  bool same(int u, int v) {
    if(root(u) == root(v)) return true;
    return false;
  }
};

int N, Q;
int Query[100009], u[100009], v[100009];
int main() {
  cin >> N >> Q;
  for(int i = 0; i < Q; i ++) cin >> Query[i] >> u[i] >> v[i];
  UnionFind UF;
  UF.init(N);
  for(int i = 0; i < Q; i ++) {
    if(Query[i] == 1) UF.unite(u[i], v[i]);
    if(Query[i] == 2) {
      if(UF.same(u[i], v[i]) == true) puts("Yes");
      else puts("No");
    }
  }
}
```

而如果你想要路径压缩呢？你就只需要将 root 函数修改一下：

```cpp
int root(int x) {
    if(par[x] != x) {
        par[x] = root(par[x]);
        // 路径压缩：将x的父节点直接设为根节点
    }
    return par[x];
}
```

关于这两个形式的复杂度分析，我们这里就不过多讲述了，但是前者，也就是只使用 Union by Size 的并查集复杂度，你们可以自己思考一下，为什么是 $O(\log N)$的？

我们会在将来找机会再解答这个问题！

# II. 最小生成树问题(MST)

## II.I. 什么是最小生成树？

生成树就是从 $M$ 条边中选择若干条构成的、联通所有顶点的树的结构，但同一张图可能存在两种以上的生成树方案.
其中，最小生成树是所有生成树中边权总和最小的结构.
这个定义可能比较难理解，如果将顶点视作“车站”、边视为“道路建设所需成本”，则更容易建立直观认识. 以最小成本连接所有车站的方法就是我们要找的最小生成树.
接下来我们看一个例题！

给定一个具有$N$个顶点和$M$条边的图，
顶点编号从1到$N$，
边$i$是连接顶点$A_i$和$B_i$的双向边，长度为$C_i$.
请计算该图的 最小生成树 中所有边的长度和.

请各位先行思考一会，想想怎么样可以找到最小生成树呢？
提示：可以考虑我们刚刚学过的东西！